

Learning SAT Variable Assignment Policies via Proximal Policy Optimization with Curriculum Learning

Sahar Chreif Mohamed Moslemani Hasan Abbani Jad Seifeddine

Abstract

The Boolean Satisfiability Problem (SAT) is the canonical NP-complete problem and lies at the heart of formal verification, planning, and combinatorial optimization. Modern conflict-driven clause learning (CDCL) solvers rely on carefully engineered heuristics—such as VSIDS and phase saving—to guide variable selection and value assignment. In this work, we investigate whether a reinforcement learning agent can discover effective branching strategies directly from data, without hand-crafted heuristics. We formulate SAT solving as a sequential decision process over the SATLIB uf20-91 benchmark and train a Proximal Policy Optimization (PPO) agent with three progressive improvements: a shaped reward signal that provides dense per-step feedback through clause proximity and falsification penalties, an enriched observation space encoding structural features such as variable activity, polarity ratios, and clause watch counts, and a curriculum learning strategy that gradually increases problem difficulty. We present an end-to-end system comprising a Gymnasium environment, Stable-Baselines3 training pipeline, FastAPI backend, and React-based monitoring interface, designed to facilitate rapid experimentation and reproducibility.

1 Introduction

The Boolean Satisfiability Problem (SAT)—determining whether there exists a truth assignment that satisfies a given propositional formula in conjunctive normal form (CNF)—was the first problem proven to be NP-complete. Despite its worst-case intractability, SAT solvers have become indispensable tools in hardware verification, software testing, automated planning, and cryptanalysis, routinely solving industrial instances with millions of variables and clauses.

The success of modern SAT solvers owes largely to the CDCL paradigm, which combines systematic search with conflict analysis and non-chronological backtracking. Within this framework, the *branching heuristic*—the policy that selects which variable to assign next and what value to give it—has an outsized impact on solver performance. Heuristics such as VSIDS (Variable State Independent Decaying Sum) and its variants have been refined over two decades of empirical tuning, yet they remain domain-agnostic: they do not adapt to the structural regularities present in specific problem families.

This observation motivates a natural question: *can a learning-based agent discover branching strategies that exploit distributional structure in SAT instances, potentially outperforming fixed heuristics on specific problem classes?* Recent advances in deep reinforcement learning (RL) have demonstrated that learned policies can match or exceed hand-engineered strategies in domains ranging from game playing to combinatorial optimization. However, applying RL to SAT solving introduces distinct challenges. The action

space grows linearly with the number of variables, the reward signal is naturally sparse (a formula is either satisfied or not), and the combinatorial structure of CNF formulas does not map naturally to the fixed-dimensional observations that standard policy networks expect.

In this paper, we address these challenges through three complementary contributions. First, we design a *shaped reward function* that replaces the sparse terminal signal with dense per-step feedback. Our reward decomposes into clause satisfaction bonuses, clause falsification penalties applied at each assignment step rather than only at termination, and a clause proximity component that grants partial credit when an assignment moves an unresolved clause closer to satisfaction. This shaping provides the learning signal needed for PPO to make meaningful gradient updates early in training.

Second, we enrich the agent's observation beyond raw variable assignments and clause statuses. Our state representation includes variable activity scores (the count of unresolved clauses each unassigned variable participates in), polarity ratios (the fraction of positive versus negative occurrences of each variable), clause watch counts (the number of unassigned literals remaining per clause), and global progress features. These features are inspired by the information that classical CDCL heuristics implicitly or explicitly maintain, but are presented in a form that a neural policy can learn from.

Third, we introduce a *curriculum learning* strategy adapted to the SAT domain. Rather than immediately confronting the agent with full instances, we begin each episode by pre-assigning a subset of variables to their known correct values, leaving only a small number of decisions for the agent. As the agent's solve rate on this reduced problem exceeds a promotion threshold, we progressively decrease the number of pre-assigned variables until the agent faces the complete instance. This curriculum allows the agent to first learn local assignment patterns before tackling global search.

We evaluate our approach on the SATLIB uf20-91 benchmark, a collection of 1,000 random 3-SAT instances at the phase transition (ratio of clauses to variables approximately 4.3), where instances are known to be satisfiable but maximally difficult for random search. Our experimental infrastructure—comprising a Gymnasium environment, Stable-Baselines3 PPO implementation, FastAPI server, and React monitoring dashboard—is designed for reproducibility and rapid iteration. We provide detailed evaluation scripts that track solve rates, clause satisfaction distributions, reward decompositions, and per-instance difficulty, enabling principled comparison across ablation conditions.

The remainder of this paper is organized as follows. Section 2 reviews related work on learning-based approaches to combinatorial optimization and SAT solving. Section 3 formalizes the SAT assignment problem as a Markov Decision Process and describes our environment, reward shaping, observation design, and curriculum strategy. Section 4 presents experimental results comparing our progressive improvements against baseline agents. Section 5 discusses limitations and directions for future work, including scaling to larger instances and incorporating graph neural network architectures.